

Mindanao State University - Iligan Institute of Technology
College of Computer Studies
Department of Computer Applications
BCA182 Embedded Systems Programming

Laboratory Activity No. 1
Real-Time Multisensor Room Monitoring System
Laboratory Report

Development Environment: PlatformIO, STM32Cube, FreeRTOS, Wokwi
Date: September 28, 2026

Shanice Reih H. Tanque

B182/B187

1. Problem and Requirements

The project implements a simulated STM32 room-monitoring system using the STM32F103C8T6 Blue Pill, STM32Cube HAL, FreeRTOS, PlatformIO, and Wokwi. The system monitors temperature, humidity, ambient light, and motion while providing an OLED user interface, rotary-encoder navigation, and a buzzer-based temperature alarm.

The required functional behavior is implemented through five concurrent FreeRTOS application tasks. Temperature and humidity are read from a DHT22, ambient light is sampled through the ADC from an LDR, and motion is detected using a PIR sensor. The OLED displays one selected measurement at a time, and the rotary encoder changes the active display page.

The temperature alarm treats values below 18 °C as LOW, values from 18 °C through 30 °C as NORMAL, and values above 30 °C as HIGH. The system starts ACTIVE, becomes INACTIVE after 15 seconds without motion, blanks the OLED while inactive, ignores encoder navigation in that state, and returns to ACTIVE when motion is detected.

Because the laboratory was completed without physical hardware, the full system is simulated in Wokwi. The project also contains Wokwi-specific compatibility adjustments, including a custom FreeRTOS Cortex-M3 port that uses TIM3 as the RTOS time base and avoids the SVC/PendSV switching path that was unreliable in the local STM32F103 simulation.

2. System Architecture and Design

2.1 Hardware Architecture

Component	Purpose / Connection
STM32F103C8T6 Blue Pill	Main controller
DHT22 on PB0	Temperature and humidity
LDR on PA0	Ambient-light ADC input
SSD1306 on PB6/PB7	I2C OLED display, address 0x3C
Rotary encoder on PA1/PA2/PA3	CLK, DT, and switch input
Buzzer on PA8	PWM alarm output
PIR on PB1	Motion input
UART1 on PA9/PA10	Serial diagnostics

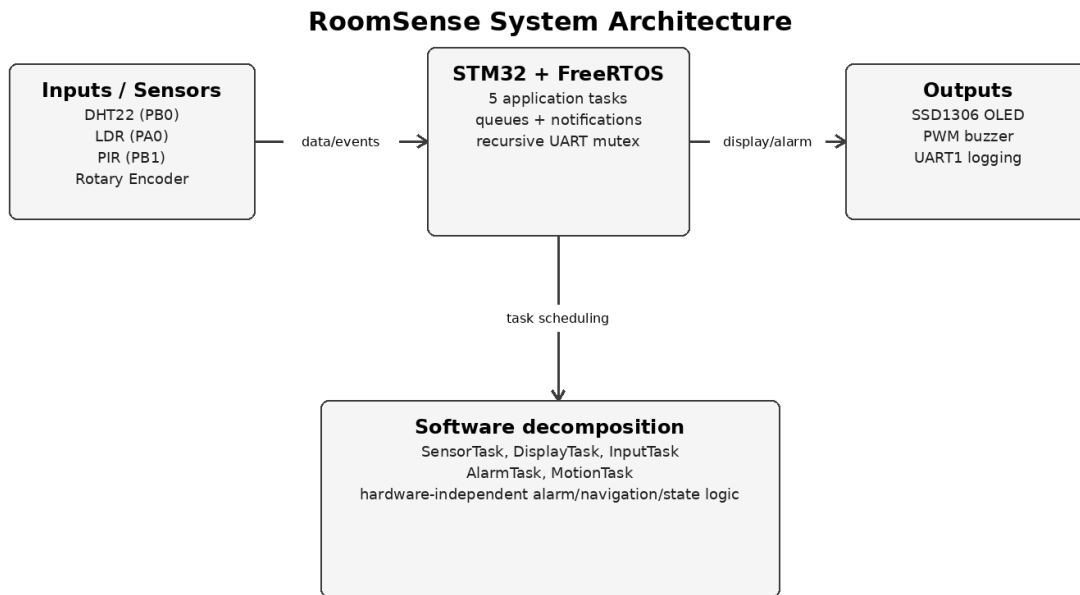


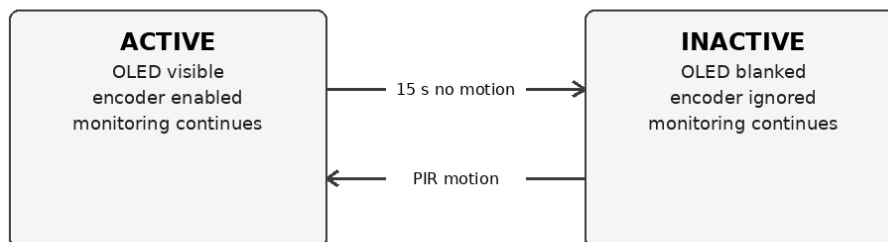
Figure 1. System architecture of the RoomSense monitor.

2.2 Software Decomposition

The firmware is divided into modules for sensing, display control, encoder input, motion/state management, alarm logic, UART logging, and RTOS communication objects. Hardware-independent functions such as temperature evaluation, display navigation, and state evaluation are kept separate from the hardware drivers so they can be unit tested in PlatformIO's native environment.

2.3 State Machine

ACTIVE / INACTIVE State Machine



MotionTask evaluates the state every 100 ms using evaluateSystemState().

Any detected motion immediately returns the system to ACTIVE.

Figure 2. ACTIVE and INACTIVE operating states.

State evaluation is performed by MotionTask using evaluateSystemState(). Any detected motion keeps or returns the system to ACTIVE. If no motion is present for at least 15 seconds, the state changes to INACTIVE. MotionTask then notifies DisplayTask of ACTIVE, INACTIVE, or MOTION events.

3. FreeRTOS Architecture

The application uses five meaningful tasks. The optional separate StateTask is not used; instead, MotionTask owns the inactivity timer and calls the hardware-independent state function. This keeps the design within the laboratory requirement of at least five meaningful tasks while avoiding an extra task whose only role would be state bookkeeping.

Task	Responsibility	Trigger Period /	Priority	IPC	Typical Blocked Condition
SensorTask	Read DHT22 and LDR; publish latest sample	2 s periodic	2	2 queues	vTaskDelayUntil()
DisplayTask	Own OLED and render selected page	Data notification driven /	1	2 queues + notification	Wait / short delay
InputTask	Process rotary encoder navigation	50 ms polling + EXTI delta	3	displayModeQueue	vTaskDelay()
AlarmTask	Evaluate temperature and control buzzer	Sensor update	2	alarmSensorQueue + notification	Queue receive
MotionTask	Read PIR and manage ACTIVE/INACTIVE	100 ms periodic	3	task notification + shared state	vTaskDelayUntil()

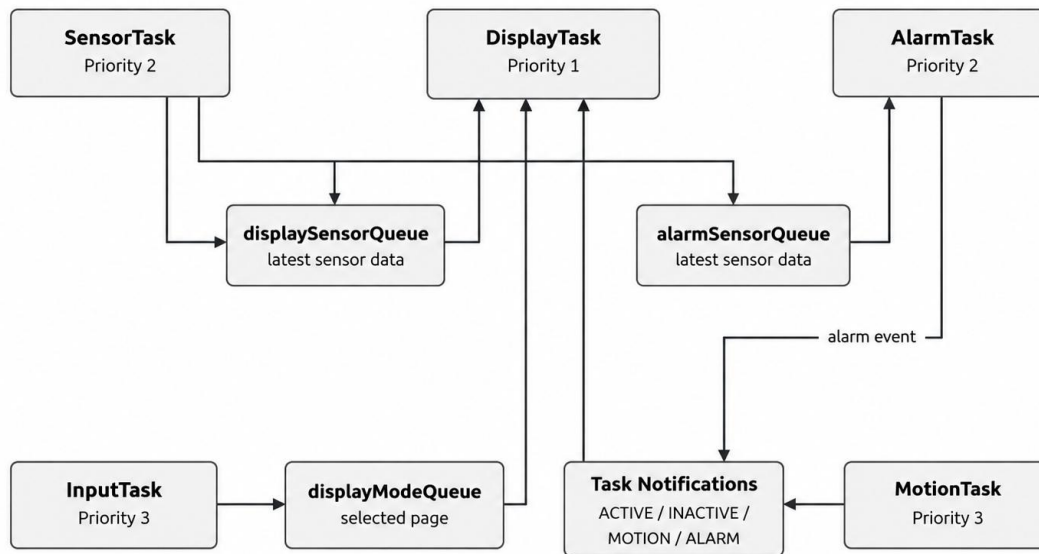
3.1 Priority and Blocking Rationale

InputTask and MotionTask use priority 3 because user input and motion state changes require prompt response. SensorTask and AlarmTask use priority 2 because environmental sampling and alarm evaluation are important but less latency-sensitive. DisplayTask uses priority 1 because OLED rendering can tolerate the most delay. The higher-priority tasks deliberately block between iterations so lower-priority tasks still receive CPU time.

3.2 Communication and Synchronization

- displaySensorQueue carries the latest SensorData from SensorTask to DisplayTask.
- alarmSensorQueue carries valid temperature samples from SensorTask to AlarmTask.
- displayModeQueue carries the selected OLED page from InputTask to DisplayTask.
- Task notifications deliver lightweight ACTIVE, INACTIVE, MOTION, and ALARM events to DisplayTask.
- serialMutex is a recursive mutex used by Log() to serialize access to UART1.
- Critical sections protect the small shared motion/state values maintained in rtos_objects.cpp.

FreeRTOS Task and Communication Architecture



All tasks use Log() through a recursive serial mutex for UART1.

Figure 3. FreeRTOS tasks, queues, notifications, and UART synchronization.

4. Implementation

4.1 Sensor Acquisition

SensorTask runs every 2 seconds using `vTaskDelayUntil()`. It reads the DHT22, converts the raw LDR ADC value to a relative percentage, obtains the current motion flag, logs the sample, and overwrites the single-element display queue with the newest reading. Only valid DHT22 temperature samples are sent to AlarmTask.

4.2 Display and Navigation

DisplayTask is the single owner of the SSD1306 OLED. InputTask processes rotary-encoder movement and writes the selected DisplayMode to displayModeQueue. Clockwise movement advances through Temperature, Humidity, Light, and Motion pages, while counterclockwise movement moves in reverse with wraparound. Encoder activity is ignored while the system is INACTIVE.

4.3 Alarm Handling

AlarmTask waits for temperature data, evaluates the temperature using `evaluateTemperature()`, controls the PWM buzzer, and logs LOW, NORMAL, or HIGH status. The alarm is active below 18 °C and above 30 °C; exactly 18 °C and exactly 30 °C are considered NORMAL. A notification is sent to DisplayTask when an alarm condition becomes active.

4.4 Motion and Activity State

MotionTask samples the PIR every 100 ms with `vTaskDelayUntil()`. Detected motion resets the inactivity timer. The task evaluates the current state, stores the latest motion and state values, and sends state-change

notifications to DisplayTask. After 15 seconds without motion the system becomes INACTIVE; subsequent PIR motion immediately restores ACTIVE.

4.5 Wokwi Adaptation

The current project includes a Wokwi-compatible FreeRTOS port for the STM32F103 simulation. TIM3 provides the 20 Hz FreeRTOS tick. The port performs direct thread-mode yields and avoids relying on the normal SVC/PendSV context-switch path that previously caused problems in the simulator. The OLED initialization also contains an APB1/I2C clock workaround specific to the Wokwi Blue Pill environment.

5. Verification and Testing

5.1 Unit Testing

Hardware-independent logic is tested using the PlatformIO native environment and the Unity framework. The project contains the 13 meaningful tests required by the laboratory: five alarm tests, four navigation tests, and four state-machine tests. The native test run completed successfully with all 13 test cases passing.

Category	Coverage	Recorded Result
Temperature alarm	Below 18, exactly 18, normal, exactly 30, above 30	5/5 passed
Display navigation	Next, previous, forward wraparound, reverse wraparound	4/4 passed
System state	Before timeout, at timeout, after timeout, motion reactivation	4/4 passed

5.2 Wokwi Functional Verification

ID	Stimulus	Observed Result	Status
FT-01	Temperature = 27.3 °C	OLED and Serial updated to 27.3 °C	PASS
FT-02	Humidity = 77.0%	OLED humidity page updated to 77.0%	PASS
FT-03	LDR illumination changed	Relative light reading changed to 10%	PASS
FT-04	Encoder CW from LIGHT	Display changed to MOTION	PASS
FT-05	Encoder CCW from MOTION	Display changed to LIGHT	PASS
FT-06	Temperature = 35.8 °C	HIGH alarm reported; buzzer ON	PASS
FT-07	Temperature = 23.9 °C	NORMAL reported; buzzer OFF	PASS
FT-08	PIR triggered while ACTIVE	Motion detected; system stayed ACTIVE	PASS
FT-09	15 s without motion	System INACTIVE; OLED blanked	PASS
FT-10	PIR triggered while INACTIVE	System returned ACTIVE; OLED restored	PASS

All ten required functional verification tests are recorded as PASS in docs/functional-verification.md.

5.3 Deliberate FreeRTOS Fault Experiments

Experiment	Observed Behavior	Engineering Interpretation
Remove InputTask blocking	Serial output flooded; normal lower-priority task output was no longer observed during the captured period.	A high-priority task that never blocks can monopolize CPU time and starve lower-priority tasks.
Raise InputTask priority 3 → 4	System continued operating; no obvious starvation was observed.	High priority alone did not fail because InputTask still blocked after each poll.
Remove UART mutex	No visible interleaving occurred in the observed run.	The run did not produce a collision, but synchronization guarantees were removed; mutex protection is still required.

Each temporary fault was removed after the experiment and the correct implementation was restored.

6. Static Code Analysis

Static analysis was run with PlatformIO Check / Cppcheck. The final pio check run recorded 0 HIGH findings, 0 MEDIUM findings, and 102 LOW findings, with both PlatformIO environments passing. The remaining LOW findings were reviewed as mainly style-related, including C-style cast suggestions and functions that static analysis could not recognize as being used indirectly through callbacks or interrupt handlers.

Severity	Findings	Interpretation
HIGH	0	No high-severity findings recorded
MEDIUM	0	No medium-severity findings recorded
LOW	102	Reviewed; predominantly style or indirect-callback analysis findings

Stable hardware and interrupt code was not changed solely to eliminate low-severity style warnings when the change would not improve functional correctness.

7. Engineering Discussion

7.1 Key Design Decisions

- `vTaskDelayUntil()` is used for periodic `SensorTask` and `MotionTask` execution to reduce timing drift.
- Two separate sensor queues are used so `DisplayTask` and `AlarmTask` do not compete for one queue item.
- `xQueueOverwrite()` is appropriate because only the newest sensor sample and display mode are needed.
- `DisplayTask` is the sole OLED owner, preventing concurrent I2C display access.
- Task notifications are used instead of an event group because the required events are lightweight and only need to signal `DisplayTask`.
- Alarm, navigation, and state decisions are separated from hardware access so they can be tested natively.

7.2 Challenges and Debugging

The main technical challenge was running FreeRTOS reliably in the Wokwi STM32F103 environment. The project uses a compatibility port with a TIM3 RTOS tick and direct yield mechanism. Another design challenge was preventing high-priority polling tasks from starving lower-priority work, which is why `InputTask` and `MotionTask` both block regularly.

7.3 Limitations

- Testing was performed mainly in Wokwi rather than on a physical STM32 board.
- The LDR value is a relative percentage and is not calibrated in lux.
- Temperature limits and the 15-second inactivity timeout are fixed in the current firmware.
- No historical sensor data is stored and there is no wireless communication.
- Heavy serial logging can influence timing, and real hardware may introduce noise, tolerances, and wiring effects not present in simulation.

8. Conclusion

The RoomSense project demonstrates a concurrent STM32 embedded application using FreeRTOS rather than a single-loop design. The final system separates sensing, display control, user input, alarm handling, and motion/state management into five tasks with explicit priorities and blocking behavior. Queues carry sensor and navigation data, task notifications deliver lightweight events, and a recursive mutex protects shared UART logging.

The project also demonstrates testable embedded design by separating hardware-independent decision logic from STM32 peripheral code. The recorded verification includes 13 passing unit tests, 10 passing Wokwi functional tests, three deliberate FreeRTOS fault experiments, and static analysis with no high- or medium-severity findings.

Future work should focus on physical-hardware validation, calibrated light measurement, configurable thresholds and inactivity timing, persistent data logging, and optional wireless monitoring.

9. Requirements Traceability Matrix

Requirement	Implementation	Verification Evidence
FR-01 Temperature measurement	SensorTask / DHT22	Unit logic support + FT-01
FR-02 Humidity measurement	SensorTask / DHT22	FT-02
FR-03 Ambient-light measurement	SensorTask / ADC LDR	FT-03
FR-04 Motion detection	MotionTask / PIR	FT-08
FR-05 OLED display	DisplayTask / SSD1306	FT-01 to FT-03
FR-06 Rotary encoder navigation	InputTask + display_logic	4 navigation unit tests + FT-04/FT-05
FR-07 Temperature alarm	AlarmTask + alarm logic	5 alarm unit tests + FT-06/FT-07
FR-08 Activity state	MotionTask + system_state	4 state unit tests + FT-08/FT-09/FT-10
FR-09 Automatic inactivity	MotionTask, 15 s timeout	State unit tests + FT-09
FR-10 Automatic reactivation	MotionTask / PIR notification	State unit tests + FT-10

10. Simulation Evidence

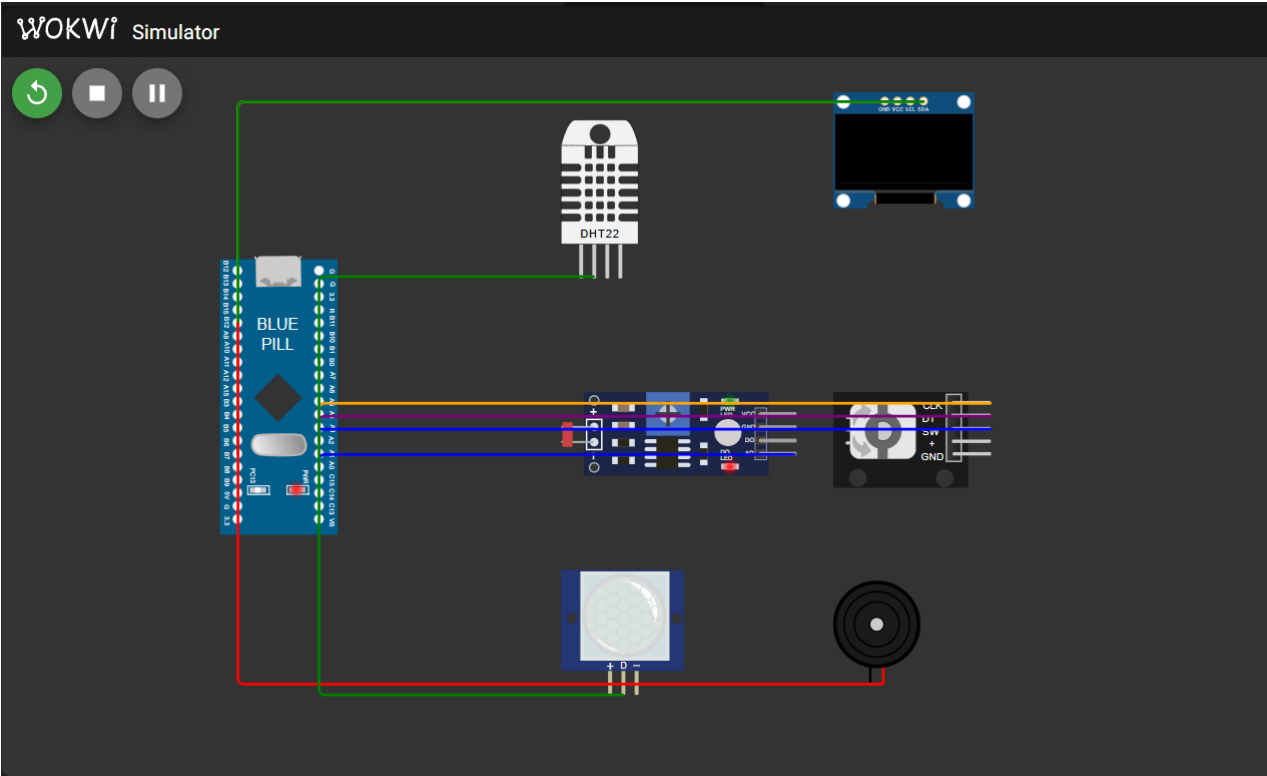


Figure 4. Wokwi circuit used for the RoomSense simulation.

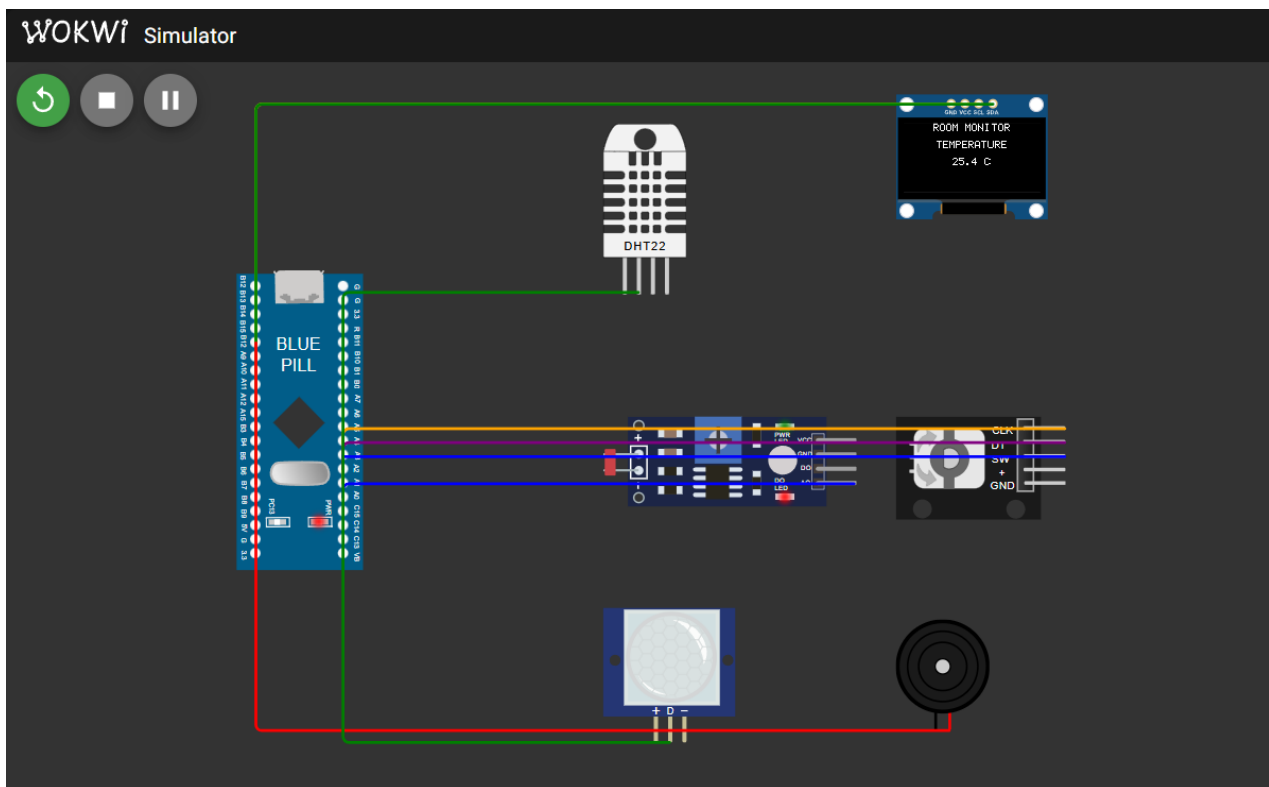


Figure 5. Finished RoomSense simulation running in Wokwi.

References

- BCA182 Embedded Systems Programming - Laboratory Activity 1.
- FreeRTOS documentation.
- STM32Cube HAL documentation.
- PlatformIO documentation.
- Wokwi documentation.
- Unity Test Framework and Cppcheck documentation.